Final Project Report

THE PROCESS

Our aircraft completed a satisfactory test flight after several attempts. We designed it to read the height from an ultrasonic sensor, gather the data from the controller to determine the desired height, and then raise or lower the craft to the defined height. On the first flight, issues arose with the P controller, as it was bouncing up and down by multiple feet. To address this, a D term was added to the code to introduce damping, thereby reducing the overall adjustment the aircraft made. However, the craft tended to drift in a rotational direction.

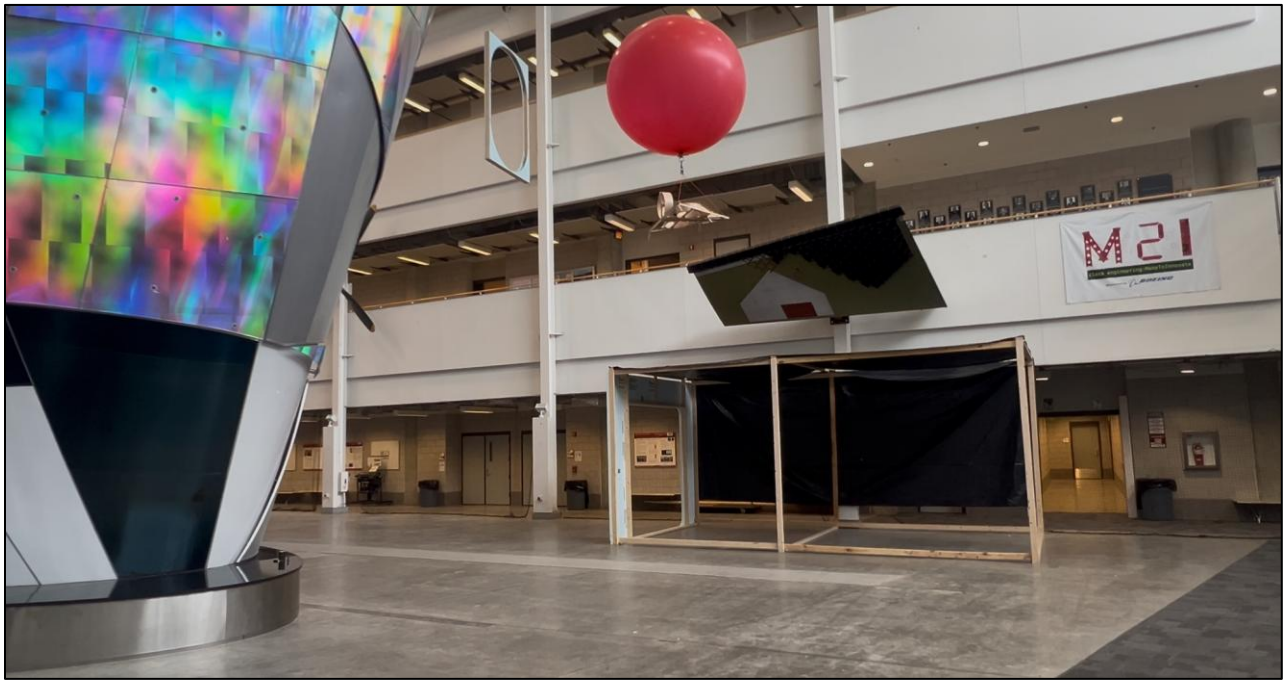


Figure 1. Picture taken during one of our 4 test flights in the Howe Hall Atrium

MODEL BUILD

To design the aircraft's structure, we followed a series of steps. First, we determined our control law and from there what propulsions we would need to make this possible. We decided on three motors, two for lift and one for thrust. Our craft was designed to be turned by applying more power to one of the two lift motors, creating an effective torque that causes rotation. With this decided, we chose a “flying wing” shape that guided the rest of our structural design. Other factors we considered were the overall weight of the craft, its ability to withstand rough impacts – which was achieved by using angled legs that provided improved stability during landing, and would not transmit the entire force through the craft if broken –, the location of the electronics, and the placement of the strings to attach to the balloon. Using this plan, we constructed the model in compliance with these factors and added support wherever necessary.

Main Structure

The main body was designed with respect to the core components as mentioned earlier. The main platform, which housed the electronics, and the central spars that ran perpendicular to it, were reinforced with wood beams underneath to reduce the effect of bending caused by the motors. Additionally, we adopted a triangular border to prevent any twisting motion during flight.

Issues and Refinements

One issue with our structure was the fact that we did not account for the space required for wire management. We managed them with tape, but next time we would add an electronic compartment to improve this management. We were, however, still able to place the electronics between the central lifting fans, creating our desired center of gravity. Another issue was that our shrouds continued to crack; this could have been prevented by utilizing a larger overlap when creating them.

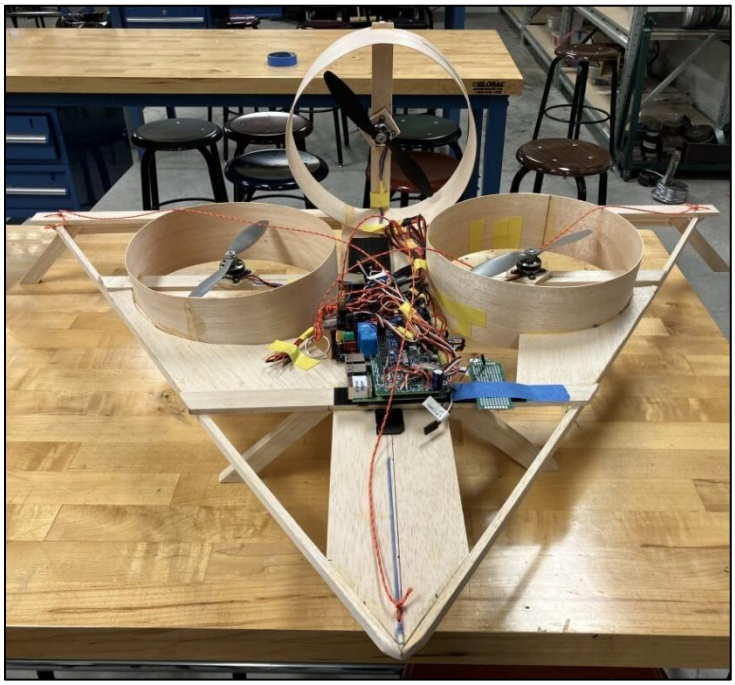


Figure 2. Final design with all of the electronics.

SOFTWARE CONTROL LAW

The software implemented a control law to determine the speed of the lift motors. It read the value of the ultrasonic sensor over serial and compared that to the desired height read from the pulse width collected from the receiver. We multiplied that error by our p term to get the pulse output to the motors. To turn left and right, we added the left/right pulse to the lift pulse.

The software got significantly more complex when we added a D controller. The ultrasonic sensor didn't allow us to directly read the derivative term, so we had to calculate it. However, because our loop ran ~20 times per second, the values we determined for the derivative term were zero. To fix this, we had to implement a queue to keep track of the last 20 values so that we could get the values 20 times ago so the difference would be measurable.

Code

Our code works like this:

1) The simulator connects the Raspberry Pi to the PWM board
2) Sets the correct timing for motor control signals
3) A startup signal is sent so the motors can arm properly
4) Throttle commands are converted into motor control signals
5) The signals are repeatedly sent to control the motors

Control Law

```
while True: pulse_us = 1500 # Send 1500 µs (neutral)

    duty = us_to_duty(pulse_us)

    motor_left.duty_cycle = duty

    motor_right.duty_cycle = duty

    print("Outputting pulse:", pulse_us, "us → duty:", duty)

    time.sleep(0.1)
```

1) Reads the distance from the ultrasonic sensor
2) The error is calculated by comparing the desired height to the actual height
3) The P term responds to the size of the error
4) The D term responds to how fast the error is changing, reducing overshoot
5) The combined control output is converted into updated throttle values
6) Sends updated throttle values to the motors

CHALLENGES

The first challenge our team encountered was getting the lift fans to respond to the pulses from the Pi. We found that the ESCs required proper initialization before the motor would response. Once this issue was resolved, we discovered that one of the motors was lacking power and had to be replaced. Additionally, we accidentally had both propellers spinning in the same direction, resulting in unwanted yaw during our hang test.

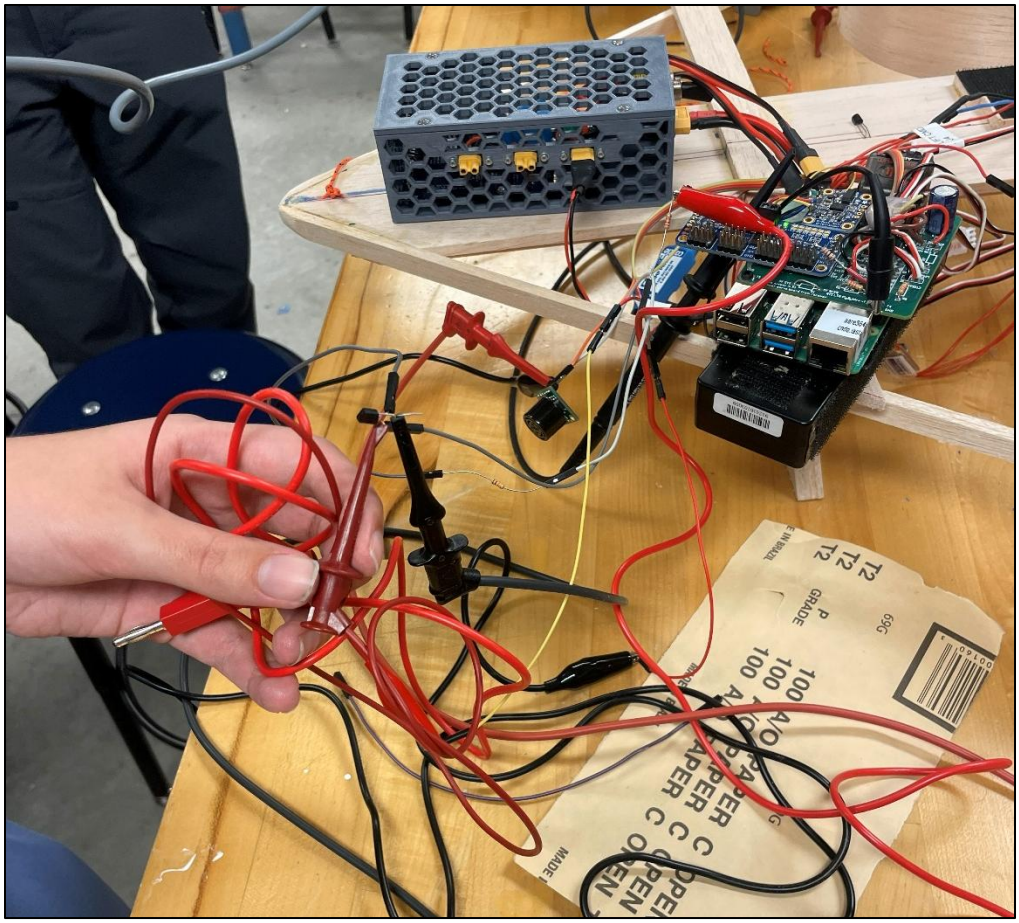


Figure 3. Testing a NOT gate to make sure it would work to invert the pulses

Our biggest challenge was that the ultrasonic sensor was sending out an inverted pulse. When we first tested it, we were getting garbage data. Only once we attached it to the oscilloscope did we see that the garbage data was due to the pulses being inverted. We had to rewire the electronics by hooking up a NOT gate to the ultrasonic sensor's TX pin, so that our Pi would receive the correct pulses and make them readable. This was accomplished by using a transistor as we did in Lab 2. If the sensor had been tested earlier, the method of applying it to the aircraft might have been different. However, the current method involved securing it with a piece of tape & Velcro due to the perfboard with the not gate attached.

During testing, we also found that the ultrasonic sensor could not function over the carpet, as the ultrasonic waves are absorbed and dissipated by the carpet. This resolved what we thought was an issue with the sensor, but it was actually an issue with the testing location.

SIMULATION

We developed a simulation code to support testing and troubleshooting of the control system. However, we did not capitalize on this opportunity as much as we should have. The simulation code read from the ultrasonic sensor, calculated the p controller, and outputted a duty cycle to the

lift fan. It was fairly basic and didn't cover as much of our hardware troubleshooting as we would have liked. It allowed us to verify the basic system response, but it was not realistic enough to display the results observed during physical testing. With flight testing revealing this overshoot, we utilized the D term to enhance damping and mitigate the oscillations.

The primary thing it helped with was troubleshooting the development of the read_sensor() function to get the ultrasonic sensor value. In the simulation, it was found that the serial timing could end up slightly off when there was something else in the main loop, causing invalid readings, so a function needed to be created to wait until it got 'R' – the first byte of the characters sent by the sensor – before reading the actual number rather than simply the next 5 bytes.

LESSONS LEARNED

If we were to do this project again, a better use of simulation would have been a good idea, allowing us to diminish our troubleshooting.

Looking into the ultrasonic sensor earlier would have also been helpful, as we would have had more time to deal with the inversion.

Another area that could have been improved is our electronics. An electronic sled would be a nice addition to the model, making it easier to remove and replace the electronic components. A more thorough plan for electronic placement would have been a good idea.

FURTHER WORK (DOUBLE CONTROL LAW)

On Thursday, Kate attempted to implement another control law on the craft that would turn the craft to a specified heading using the gyroscope sensor, as we had done in the simulator. This venture was undertaken to fix the fact that the craft tends to rotate when flying and needs correction. A second control law would supposedly keep the craft from spinning when going up and down. Although she successfully implemented the rotational control law on a hang test alone, it proved difficult to control in combination with the altitude control law. Firstly, since the left/right and lift motors are the same, the spin correction didn't work when the craft required high lift speed. Even with bounding, the upper limits still struggled to correct the spin. The craft remained rotationally aligned at a lower height difference between the desired and actual values. However, because of the way the craft was designed, the code was switching between the left and right motors to align the rotation. When the craft had both motors on to ascend and was correcting its rotation, this caused a significant amount of back-and-forth tilt. Thus, Kate decided that this instability was too problematic to flight test this code version because it would likely flip and crash.